# TEST PLAN & TEST CASE SPECIFICATION

Traffic Management System (Simulation)

Software Engineering — Mini Project

| | |
|---|---|
| **Document** | Final corrected Test Plan + Test Case Update |
| **Implementation** | Standalone C/C++ console simulation |
| **Primary data structure** | Queues (per-lane FIFO) |
| **Test cases** | 12 test cases (TC-01 to TC-12) |
| **Reference documents** | Project SRS and Software Architecture & Design Specification (SADS) |
| **Test result status** | Actual Result and Pass/Fail fields are blank until tests are executed. |

# 1. Introduction

## 1.1 Purpose

This Test Plan defines the scope, strategy, environment, organization, and execution approach for testing the Traffic Management System (Simulation). It checks the functional requirements FR1–FR8, non-functional requirements NFR1–NFR3, security objectives SO1–SO2, and security requirements SR1–SR2 stated in the project SRS, and checks behavior against the SADS.

## 1.2 Scope

Testing covers vehicle addition, per-lane queues, traffic signal transitions, vehicle movement, traffic information display, congestion detection, statistics, performance, usability, reliability, input validation, and queue data protection. The system is treated as a standalone, single-user, offline C/C++ console simulation. Networking, authentication, encryption, multi-user operation, and persistence are outside the defined scope.

## 1.3 References

• Software Requirements Specification (SRS) — Traffic Management System (Simulation).
• Software Architecture & Design Specification (SADS) — Traffic Management System (Simulation).
• Software Engineering Unit 3, Lecture 8 slides supplied for this assignment (test cases and test design concepts).

## 1.4 Definitions

**Test case:** documented inputs, conditions, steps, and expected results used to check a requirement.
**Positive test:** checks expected behavior for valid input.
**Negative test:** checks that invalid input or an invalid operation is safely rejected.
**Destructive/limit test:** checks behavior under a deliberately demanding workload or condition, without intentionally damaging the environment.
**Actual result:** what is observed when the test is executed.
**Pass/Fail:** comparison of actual result against expected result.

## 1.5 Test Objectives

• Verify that each FR1–FR8 requirement behaves as specified.
• Check NFR1 performance, NFR2 usability, and NFR3 reliability.
• Verify input validation and queue protection for SR1/SR2 and data integrity for SO1/SO2.
• Use explicit test steps and expected results so another tester can repeat each test.
• Maintain traceability from requirements to test cases and record defects found during execution.

# 2. Test Strategy and Test Design Techniques

## 2.1 Test Levels

• **Unit testing:** check individual component behavior where practical, such as queue enqueue/dequeue and signal-state functions.
• **Integration testing:** check interaction between the Simulation Controller, Vehicle Manager, Signal Controller, Congestion Detector, Statistics Manager, and Queue Manager.
• **System testing:** run complete scenarios through the simulation and compare displayed results with expected behavior.

## 2.2 Test Design Techniques

• **Specification-based / black-box testing:** derive test cases from the SRS and expected externally visible behavior, without relying on implementation details.
• **Equivalence partitioning:** group inputs expected to behave similarly, such as valid lane identifiers versus invalid lane identifiers; test representative values from each group.
• **Boundary-value analysis:** check values at and near a boundary. TC-06 tests the congestion threshold and the value immediately above it; use the actual configured threshold.
• **Pairwise / combination testing:** check relevant interactions, such as signal state and queue status, rather than testing each parameter in isolation.
• **Negative testing:** check malformed identifiers and invalid operations such as dequeue on an empty queue.
• **Limit/workload testing:** exercise a larger representative number of vehicle operations and observe whether the application remains operational. The SRS does not define a numerical response-time target, so none is invented here.

## 2.3 Testing Principles Applied

• Testing can reveal defects, but passing tests do not prove that no defects exist.
• Exhaustive testing is impractical because input combinations and operation sequences can be very large; select tests based on requirements, boundaries, invalid cases, and relevant combinations.
• Test early when components and requirements are available, so defects can be found before final submission.
• Give attention to complex or frequently changed components and to areas where defects have already been observed.
• Vary test inputs and scenarios instead of repeatedly running only one unchanged test set.
• Testing is context-dependent: this plan focuses on traffic queues, signal transitions, congestion, statistics, and safe handling of invalid operations.
• **Verification** checks conformance to the stated requirements; **validation** checks whether the simulation supports the intended traffic-management use.

## 2.4 Test Case Types

Each test case is classified as positive, negative, or destructive/limit where applicable. Positive cases use valid inputs; negative cases use invalid inputs or invalid states; destructive/limit cases apply a demanding but controlled workload to observe safe behavior. A case may have more than one purpose.

## 2.5 Test Case Record Format

Each case below includes the lecture-slide parameters: Test Case ID, Test Scenario/Title, Test Case Description, Test Setup, Test Steps, Prerequisites, Test Parameters, Expected Result, Actual Result, and Comments. A test type and requirement mapping are also included to make coverage easier to review.

## 2.6 Environment and Tools

• Platform: workstation capable of compiling and running the standalone C/C++ console simulation.
• Tools: GCC/G++ or equivalent compiler, terminal/console, source editor/IDE, and this test-case record.
• Execution: single-process, single-thread simulation with no network exposure.

# 3. Test Cases and Expected Results

The following 12 test cases retain the project's existing requirement IDs and scope. Each case uses the lecture-slide fields. Test type is identified as positive, negative, or destructive/limit where appropriate. Actual Result and Pass/Fail remain blank until execution.

## TC-01 — Add a vehicle using valid input

| | |
|---|---|
| **Test Case ID** | TC-01 |
| **Test Scenario / Title** | Add a vehicle using valid input |
| **Test Case Description** | Verify that a vehicle can be added to a valid configured lane using a valid vehicle identifier. |
| **Test Setup** | Build and start the simulation; confirm configured lane identifiers. |
| **Test Steps** | 1. Submit addVehicle(validLaneId, V1).<br>2. Observe the response.<br>3. Query the lane queue length and/or display traffic information. |
| **Prerequisites** | Simulation is running; target lane exists and is initialized. |
| **Test Parameters** | laneId = one valid configured lane (e.g., Lane A if configured); vehicleId = V1. |
| **Expected Result** | The input is accepted; V1 is created and enqueued in the selected lane; the output confirms success; queue length increases by one. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Positive, specification-based test. Use an actual configured lane identifier. |
| **Test Type** | Positive |
| **Requirement ID(s)** | FR1, SR1 |

## TC-02 — Verify FIFO queue order and removal

| | |
|---|---|
| **Test Case ID** | TC-02 |
| **Test Scenario / Title** | Verify FIFO queue order and removal |
| **Test Case Description** | Verify that vehicles are maintained in FIFO order and that one valid movement/removal removes the front vehicle only. |
| **Test Setup** | Start simulation and initialize a valid lane. |
| **Test Steps** | 1. Add V1, V2, V3 in that order.<br>2. Query queue length.<br>3. Trigger one valid move/remove operation.<br>4. Inspect queue order and length. |
| **Prerequisites** | Target lane exists; queue can accept vehicles. |
| **Test Parameters** | Add V1, then V2, then V3 to the same lane; expected queue order = V1, V2, V3. |
| **Expected Result** | Queue length is 3 before removal; V1 is removed first; V2 and V3 remain in order; queue length becomes 2. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Positive test and queue-integrity check. |
| **Test Type** | Positive |
| **Requirement ID(s)** | FR2, FR5, NFR3 |

## TC-03 — Verify the traffic signal cycle

| | |
|---|---|
| **Test Case ID** | TC-03 |
| **Test Scenario / Title** | Verify the traffic signal cycle |
| **Test Case Description** | Verify that the signal moves only through the defined Red, Yellow, and Green cycle. |
| **Test Setup** | Start simulation and initialize a lane's signal to a known state. |
| **Test Steps** | 1. Record the starting signal state.<br>2. Call updateSignal() repeatedly for a complete cycle.<br>3. Record the state after each call. |
| **Prerequisites** | Signal Controller is initialized and its cycle rules are available in the project specification. |
| **Test Parameters** | Input: repeated updateSignal() calls; observation: getSignalState(laneId) after each update. |
| **Expected Result** | Signal transitions follow the configured cycle and only the permitted Red, Yellow, and Green states are observed. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Positive, specification-based state-transition test. Follow the exact cycle order defined by the implementation/SRS. |
| **Test Type** | Positive |
| **Requirement ID(s)** | FR3, FR4 |

## TC-04 — Verify vehicle movement depends on signal state

| | |
|---|---|
| **Test Case ID** | TC-04 |
| **Test Scenario / Title** | Verify vehicle movement depends on signal state |
| **Test Case Description** | Verify that a queued vehicle does not move on Red or Yellow and can move when the lane signal reaches Green, according to the specified rules. |
| **Test Setup** | Start simulation with a known signal state and one vehicle queued. |
| **Test Steps** | 1. Add V1 while the signal is Red or Yellow.<br>2. Observe the queue during Red/Yellow.<br>3. Advance the normal signal cycle to Green.<br>4. Observe vehicle movement and queue length. |
| **Prerequisites** | Target lane exists and contains at least one vehicle. |
| **Test Parameters** | Vehicle = V1; signal states to observe = Red, Yellow, Green. |
| **Expected Result** | V1 remains queued during Red/Yellow; when the signal reaches Green, movement follows the defined simulation rules and the queue is updated once. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Positive integration test across Signal Controller, Simulation Controller, and Vehicle Manager. |
| **Test Type** | Positive |
| **Requirement ID(s)** | FR3, FR5 |

## TC-05 — Check traffic information displayed to the user

| | |
|---|---|
| **Test Case ID** | TC-05 |
| **Test Scenario / Title** | Check traffic information displayed to the user |
| **Test Case Description** | Verify that displayed traffic information accurately shows lane, signal state, and waiting vehicle count. |
| **Test Setup** | Start simulation and prepare known queue and signal states. |
| **Test Steps** | 1. Set up the known state using valid operations.<br>2. Trigger displayTrafficInfo().<br>3. Compare the output with the known state. |
| **Prerequisites** | At least one lane has a known signal state and known queue length. |
| **Test Parameters** | Example state: lane contains V1 and V2; queue length = 2; signal state = the current configured state. |
| **Expected Result** | The output shows the lane identifier, current signal state, and waiting vehicle count; displayed values match the simulation state and are understandable. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Positive test; uses a checklist for consistent review. |
| **Test Type** | Positive |
| **Requirement ID(s)** | FR6, NFR2 |

## TC-06 — Check congestion at and above the threshold

| | |
|---|---|
| **Test Case ID** | TC-06 |
| **Test Scenario / Title** | Check congestion at and above the threshold |
| **Test Case Description** | Verify the exact congestion boundary: a lane is congested only when queue length exceeds the configured threshold. |
| **Test Setup** | Find the congestion threshold T from the actual configuration or project specification. |
| **Test Steps** | 1. Set queue length to T-1 and call detectCongestion(laneId).<br>2. Repeat at T.<br>3. Repeat at T+1. |
| **Prerequisites** | Congestion threshold T is known and the lane can be populated to the required sizes. |
| **Test Parameters** | Queue lengths = T-1, T, and T+1. Do not substitute an assumed numeric threshold. |
| **Expected Result** | At T-1 and T, lane is not reported congested; at T+1, lane is reported congested, consistent with the stated 'exceeds threshold' rule. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Explicit boundary-value analysis. If the SRS defines a different comparison rule, follow the SRS and update expected results accordingly. |
| **Test Type** | Positive / Boundary-value |
| **Requirement ID(s)** | FR7 |

## TC-07 — Verify a lane below the congestion threshold

| | |
|---|---|
| **Test Case ID** | TC-07 |
| **Test Scenario / Title** | Verify a lane below the congestion threshold |
| **Test Case Description** | Verify that a lane below the congestion threshold is not incorrectly reported as congested. |
| **Test Setup** | Find the configured threshold T and prepare a lane with fewer than T vehicles. |
| **Test Steps** | 1. Add vehicles until the queue length is below T.<br>2. Call detectCongestion(laneId).<br>3. Record the returned status. |
| **Prerequisites** | Lane exists; threshold is configured and known. |
| **Test Parameters** | Queue length = T-1 (or another valid count below T when T is too small for T-1). |
| **Expected Result** | The lane is reported as not congested. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Negative/under-threshold test. Complements TC-06. |
| **Test Type** | Negative / Boundary support |
| **Requirement ID(s)** | FR7 |

## TC-08 — Verify traffic statistics

| | |
|---|---|
| **Test Case ID** | TC-08 |
| **Test Scenario / Title** | Verify traffic statistics |
| **Test Case Description** | Verify that statistics reflect the recorded vehicle additions and movements. |
| **Test Setup** | Start simulation and prepare a known sequence of vehicle operations. |
| **Test Steps** | 1. Record the initial state.<br>2. Add V1, V2, V3.<br>3. Perform a known number of valid movements.<br>4. Call calculateStatistics().<br>5. Compare reported statistics with recorded events. |
| **Prerequisites** | Statistics function is available; operations can be observed or recorded. |
| **Test Parameters** | Add V1, V2, V3; perform the defined number of valid movement/removal operations; compare reported counts with the observed events. |
| **Expected Result** | Reported vehicles processed and other defined statistics match the valid events and the project's specified calculation rules. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Positive test. Do not assume a formula for waiting time unless it is defined by the SRS/SADS. |
| **Test Type** | Positive |
| **Requirement ID(s)** | FR8 |

## TC-09 — Exercise a representative high workload

| | |
|---|---|
| **Test Case ID** | TC-09 |
| **Test Scenario / Title** | Exercise a representative high workload |
| **Test Case Description** | Check whether a representative larger workload can be processed without crash, hang, freeze, or failure. |
| **Test Setup** | Start the simulation in a clean state; select a workload that is practical for the actual application and environment. |
| **Test Steps** | 1. Record the workload size and environment.<br>2. Add the selected number of vehicles across valid lanes.<br>3. Run signal cycles and valid movement/statistics operations.<br>4. Observe application responsiveness and completion. |
| **Prerequisites** | Simulation is running and ready to accept valid operations. |
| **Test Parameters** | A documented, representative larger number of vehicles across configured lanes; repeated signal updates, valid moves, and statistics calls. Record the chosen count during execution; no numeric performance threshold is defined in the SRS. |
| **Expected Result** | The selected workload completes without crash, hang, freeze, or operation failure; the console remains usable. Record the workload size and any observed timing, without treating an undefined timing target as a requirement. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Controlled limit/workload test, not destructive data deletion. Choose and record the workload before execution. |
| **Test Type** | Destructive / Limit (controlled) |
| **Requirement ID(s)** | NFR1 |

## TC-10 — Protect an empty queue from invalid removal

| | |
|---|---|
| **Test Case ID** | TC-10 |
| **Test Scenario / Title** | Protect an empty queue from invalid removal |
| **Test Case Description** | Verify that dequeue/removal on an empty queue is safely rejected or guarded and does not corrupt subsequent queue operations. |
| **Test Setup** | Start simulation and select a valid lane with an empty queue. |
| **Test Steps** | 1. Attempt removal from the empty queue.<br>2. Observe the response and queue state.<br>3. Add V1 and V2.<br>4. Perform valid removal and inspect queue contents. |
| **Prerequisites** | Target lane exists and its queue length is zero. |
| **Test Parameters** | First operation: removeVehicle(laneId) or dequeue on empty queue. Follow-up: enqueue V1 and V2, then perform valid removals. |
| **Expected Result** | Empty-queue removal is rejected/guarded; no crash or state corruption occurs; later valid operations preserve the expected FIFO order and no vehicle is lost or duplicated. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Negative test for queue protection and reliability. |
| **Test Type** | Negative |
| **Requirement ID(s)** | NFR3, SR2 |

## TC-11 — Reject invalid lane or vehicle identifiers

| | |
|---|---|
| **Test Case ID** | TC-11 |
| **Test Scenario / Title** | Reject invalid lane or vehicle identifiers |
| **Test Case Description** | Verify that malformed or invalid lane/vehicle identifiers are rejected before a vehicle or queue state is modified. |
| **Test Setup** | Start simulation and identify the valid identifier format and configured lane list. |
| **Test Steps** | 1. Submit addVehicle() with an invalid lane ID.<br>2. Observe response and queue state.<br>3. If supported by the interface, repeat with a malformed vehicle ID.<br>4. Check that no invalid vehicle was added. |
| **Prerequisites** | Simulation is running; valid identifier rules are available. |
| **Test Parameters** | Use a lane identifier not present in the configured lanes and/or a vehicle identifier that violates the project's defined format. Record exact values used. |
| **Expected Result** | Input is rejected with an error or safe rejection response; no vehicle is created and no queue is modified. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Negative test; use invalid values derived from the project's actual validation rules. |
| **Test Type** | Negative |
| **Requirement ID(s)** | SR1, SO2 |

## TC-12 — Check data integrity across mixed operations

| | |
|---|---|
| **Test Case ID** | TC-12 |
| **Test Scenario / Title** | Check data integrity across mixed operations |
| **Test Case Description** | Verify that valid operations change the simulation state as expected while rejected invalid operations do not corrupt vehicle, queue, signal, or statistics data. |
| **Test Setup** | Start simulation and prepare at least one lane with known queue contents and signal state. |
| **Test Steps** | 1. Record the initial vehicle, queue, signal, and statistics state.<br>2. Perform a valid add/move sequence.<br>3. Attempt the invalid operations.<br>4. Inspect final state and compare against the expected state derived from valid operations only. |
| **Prerequisites** | Known initial state can be recorded and compared with final state. |
| **Test Parameters** | Valid operations: add V1 and perform a permitted move. Invalid operations: one invalid lane/vehicle input and one removal from an empty queue, where supported. |
| **Expected Result** | Invalid operations have no unintended effect; valid operations are reflected correctly; vehicle/queue/signal/statistics data remain consistent; no vehicle is lost or duplicated. |
| **Actual Result** | |
| **Pass / Fail** | |
| **Comments** | Combined integration and data-integrity test for SO1, SO2, SR1, and SR2. |
| **Test Type** | Negative / Integration |
| **Requirement ID(s)** | SO1, SR1, SR2 |

## 5.1 Security Validation

This section explicitly checks the security objectives SO1 (Data Integrity) and SO2 (Input Validation), and security requirements SR1 (Input Validation) and SR2 (Queue Data Protection). The cases below are TC-10, TC-11, and TC-12; execute them and record actual results.

| ID | Requirement | Test case | Validation activity | Expected result |
|---|---|---|---|---|
| SR1 | Validate vehicle/simulation inputs before processing. | TC-11 | Submit invalid lane/vehicle identifiers and inspect response and queue state. | Invalid input is rejected; no vehicle or queue state is created or modified. |
| SO2 | Ensure input validation prevents invalid data from affecting the simulation. | TC-11 | Confirm rejected input does not reach vehicle/queue processing. | Simulation state remains unchanged by rejected input. |
| SR2 | Prevent invalid queue operations from losing, duplicating, or corrupting vehicles. | TC-10 | Attempt removal from an empty queue and then perform valid operations. | Invalid operation is guarded; queue order and contents remain correct. |
| SO1 | Maintain integrity of vehicle, queue, signal, and statistics data. | TC-12 | Mix valid and invalid operations and compare final state with expected state. | Invalid operations do not corrupt state; data remain consistent. |

### 5.2 Execution and Defect Handling

For each test: (1) prepare the required state; (2) perform the documented steps; (3) observe the output/state; (4) compare actual and expected results; (5) mark Pass or Fail; and (6) log failures with test case ID, requirement ID, steps, expected result, actual result, and severity. After a fix, rerun the failed test and relevant regression tests.

Actual Result and Pass/Fail fields have intentionally not been pre-filled because no execution evidence was supplied with the source document. Fill them in only after running the test cases.

# 6. SRS Traceability Matrix

This matrix maps the requirements listed in the existing project Test Plan to the test cases that verify them.

| Requirement ID | Requirement summary | Test Case ID(s) |
|---|---|---|
| FR1 | Add vehicles to the simulation | TC-01, TC-11 |
| FR2 | Maintain per-lane queues and queue length | TC-02 |
| FR3 | Transition signals through the defined cycle | TC-03, TC-04 |
| FR4 | Return/display valid signal states | TC-03 |
| FR5 | Allow vehicles to enter/leave according to traffic rules | TC-02, TC-04 |
| FR6 | Display signal state, queue information, and waiting counts | TC-05 |
| FR7 | Detect congestion using queue-length threshold | TC-06, TC-07 |
| FR8 | Calculate traffic statistics | TC-08 |
| NFR1 | Process representative workload without crash/hang/freeze | TC-09 |
| NFR2 | Provide clear and understandable traffic information | TC-05 |
| NFR3 | Maintain queue integrity without loss or duplication | TC-02, TC-10, TC-12 |
| SO1 | Maintain data integrity | TC-12 |
| SO2 | Validate inputs before processing | TC-11 |
| SR1 | Reject invalid vehicle/lane input | TC-11 |
| SR2 | Protect queues from invalid operations | TC-10, TC-12 |

# 7. Test Organization, Entry/Exit Criteria, and Submission Checklist

**Responsibilities:** The tester prepares test data, executes test cases, records results, and reports defects. Development responsibilities remain with the project team members assigned to the components.

**Entry criteria:** (1) source code compiles successfully; (2) SRS and SADS are available and unchanged; (3) configured lane identifiers, signal cycle, and congestion threshold can be identified; (4) test cases are ready to execute.

**Exit criteria:** (1) all 12 test cases have been executed at least once; (2) every requirement in the traceability matrix is mapped to at least one test case; (3) failures and defects are recorded; (4) high-severity defects are fixed and retested or explicitly documented as deferred.